Machine Learning – CDS201

Module 1: Introduction to Machine Learning

Introduction to machine learning

- What is human learning?
  - In cognitive science, learning -> Gaining information through observation

  - Why do humans learn? To carry out multiple activities
  - As we keep learning more -> the efficiency in doing the tasks keeps improving.
  - More learning -> more efficient tasks.

- Types of human learning
  - Learning under expert guidance – expert in the subject directly teaches us

  - Learning guided by knowledge gained from experts – based on our own notion gained based on our learning from experts or from past

  - Learning by self – after several unsuccessful attempts

Introduction to machine learning

- ML by Tom M. Mitchell, from CMU

- A computer program is said to learn from *experience E* with respect to some class of *tasks T* and *performance measure P*, if its performance at *tasks in T*, as *measured by P*, improves with *experience E*.

Introduction to machine learning

- Machine learning is a subset of artificial intelligence (AI) that focuses on creating algorithms that can learn from and make predictions or decisions based on data.
- Unlike traditional programming, where a system follows predefined rules, ML algorithms improve themselves by identifying patterns in vast datasets. These patterns enable systems to evolve and become more accurate over time without requiring human intervention.

Traditional coding vs Training

- Consider the classic example of classifying cat images.
- Hard-coded attempts have been made for decades by trying to extract specific tell-tale features, like ears and whiskers – but you can never be sure if these features may not be visible from the angle the picture is taken, or if a set of ears and whiskers gets identified on another animal or a plant.
- But using logic alone, it was hard to imagine how our brain solved this problem so quickly and precisely.
- Recognize cats – training the images from all angles and in all sizes - just as humans can understand and appreciate spatial reasoning, without explicitly resorting to it every time they need to identify a cat.

Introduction to machine learning

- In ML, the system learns logic from the data, while in traditional programming, the programmer needs to explicitly code the rules/logic.
- User provides data (independent variables and dependent variable) and ML algorithm learn by itself.
- ML algorithms are very useful in cases where the patterns/rules in the data are complex, hidden or so many that they cannot be explicitly coded.
- While in traditional programming, the programmer needs to manually update the logic if some conditions change, the ML algorithm can adapt to new scenarios by providing the new data and retraining the model

ML applications

- Healthcare diagnostics and predictive analytics
- Autonomous vehicles
- E-commerce and recommendation systems
- Fraud detection in finance
- Natural Language Processing (NLP)
- And many more……

Issues in Machine learning

- Data is the new OIL.
- Privacy and breach of data - Loss of trust, legal consequences, and harm to individuals.
- Emotional and Psychological Harm - Dehumanizing automated responses, Automated rejection systems. Eg: customer segmentation - Low-income areas always tagged as “low value customers”.
- Lack of accountability

Problems not to be solved using machine learning

- ML should not be applied to tasks in which there are frequent human intervention is needed.
- Example: Air Traffic Control (Rules change dynamically) ; controllers must override decisions frequently. ML should assist ATC, not replace it.
- Using ML to decide disciplinary action for students in a university & ML for real-time exam invigilation decisions
- Works better when there is a **human in the Loop.**

- Simple task like price calculator engine. A calculator is not suitable for machine learning because arithmetic follows fixed rules with exact outcomes, and any deviation would require frequent human correction.

How ML Learns

- Machine Learning does not “understand” data — it transforms vectors mathematically.

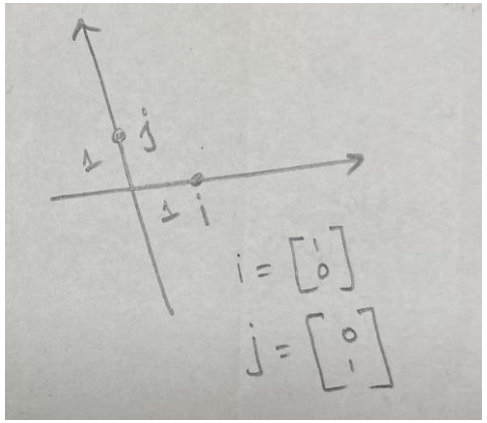
- Data -> Vectors

- In ML, real-world data (customers, voters, users) is converted into numerical vectors.

- **Example (Customer Segmentation):**
  - Age → 25
  - Income → 40,000
  - Frequency of purchase → 12
  - X=[25,40000,12]

- Basis vectors
  - Basis vectors are the reference directions using which we describe every other vector.

$$\mathbf{i} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{j} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

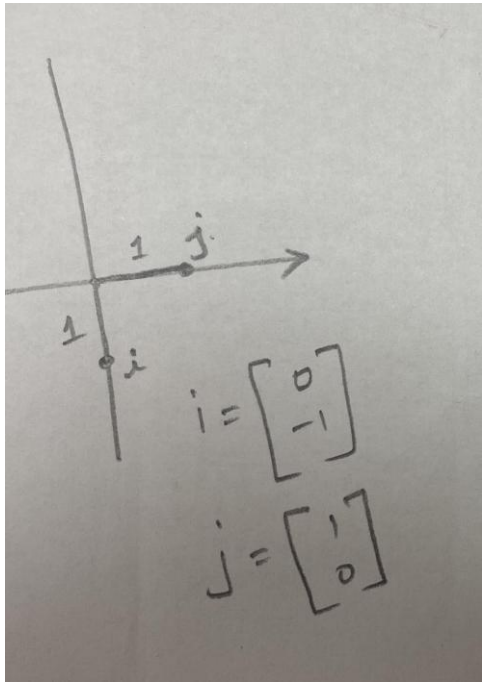


  - **i** → one unit step along x-axis
  - **j** → one unit step along y-axis
- Any vector in 2D can be written as:

$$\vec{v} = a\mathbf{i} + b\mathbf{j}$$

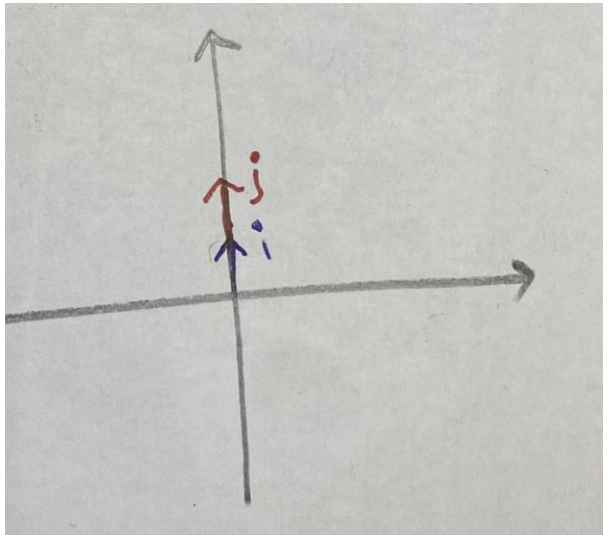
- Example: $-3\mathbf{i} + 2\mathbf{j}$
  - move **–3 units** in x direction
  - move **+2 units** in y direction

- Basis vector with 90-degree rotation in clockwise, the new transformed basis is given below.
- As I keep rotating and scaling, I can reach anywhere in the x-y 2D space → "SPAN"

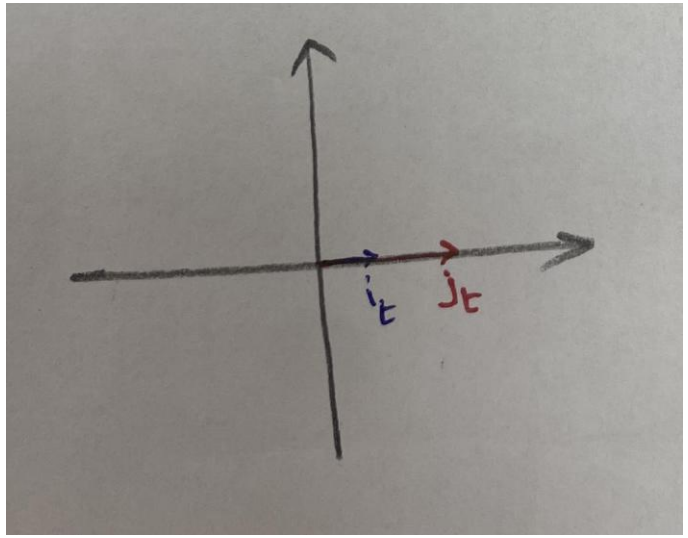


- Are **i, j** different from **x, y**? Yes
  - **i and j** — Basis Vectors (Vector space)
  - **x and y** — Coordinates (Scalars)
  - The x-axis is the line spanned by i. The y-axis is the line spanned by j

- If the basis vector are parallel to each other. The SPAN is only across the 1-dimension



90-degree clockwise rotation



Linear transformations and geometric interpretations of learning

- Linear algebra is a fundamental mathematical concept that forms the backbone of many machine learning algorithms.

- A scalar is a single value, like a number. A vector, on the other hand, is a quantity that has both magnitude and direction. In machine learning, vectors are often used to represent features of data points.

- For instance, if you're working with images, you could represent each image as a vector where each entry corresponds to a pixel value.

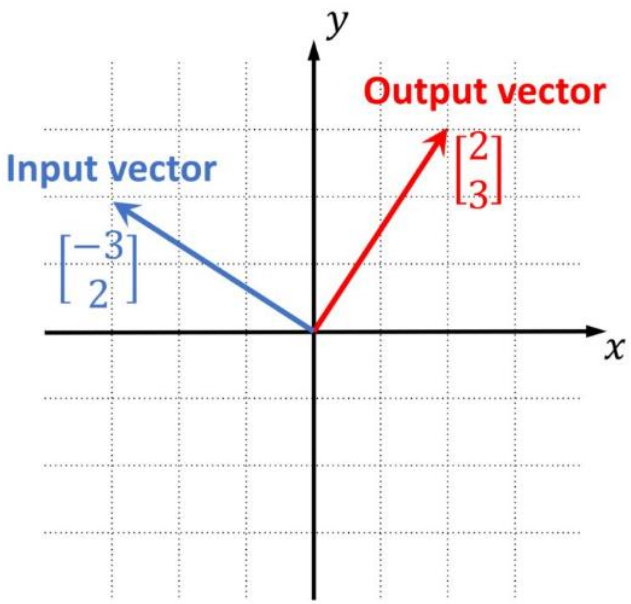
- A linear transformation can also be seen as a simple function.
- A **linear transformation is actually a function that maps an input vector into an output vector.**
- This input can be either a scalar or a vector.

- *When we apply a linear transformation, we are either transforming the vector itself or changing the basis used to describe it.*
- Rotation, scaling, projection = **new basis directions**

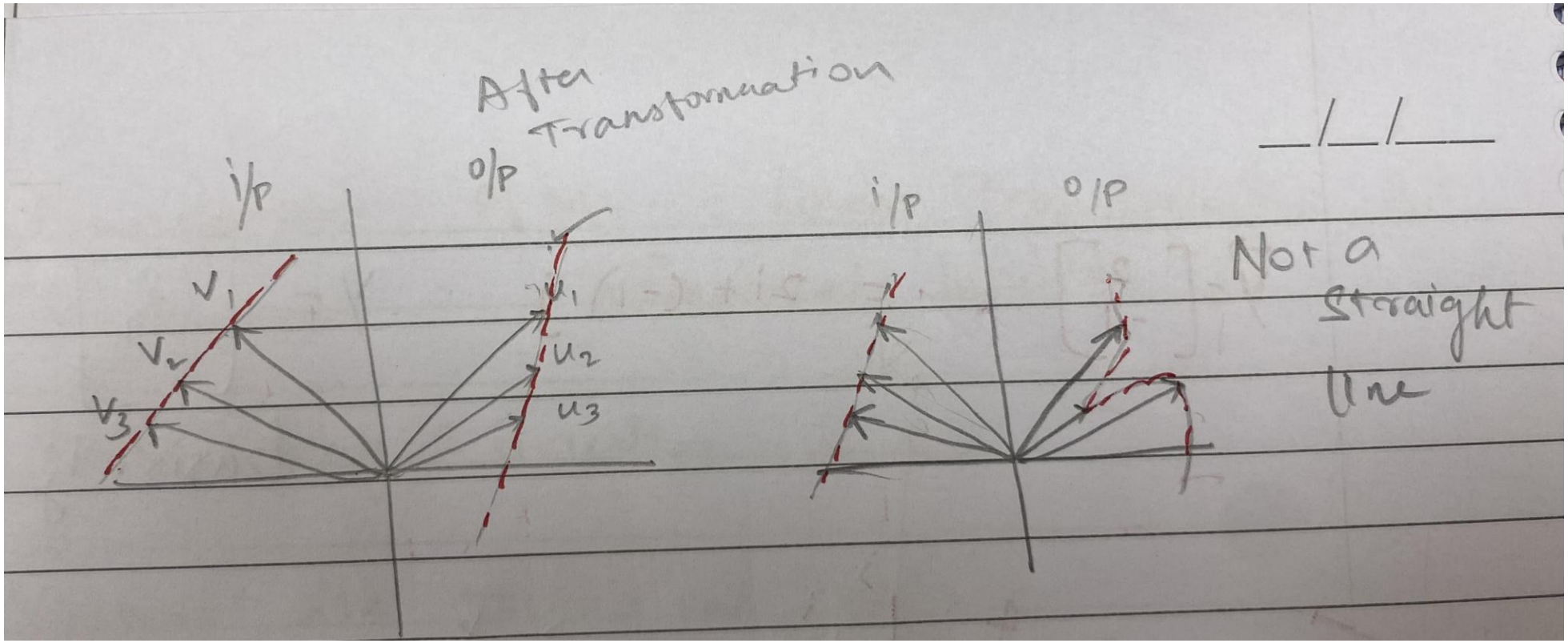
Linear transformation properties

- There are two properties that a linear transformation has to obey so that we are allowed to call it a linear transformation.
  - **A line should remain a line once we transform our coordinate system**.
  - The **origin should remain at the fixed place**

- The **origin should remain at the fixed place**



- **A line should remain a line once we transform our coordinate system**



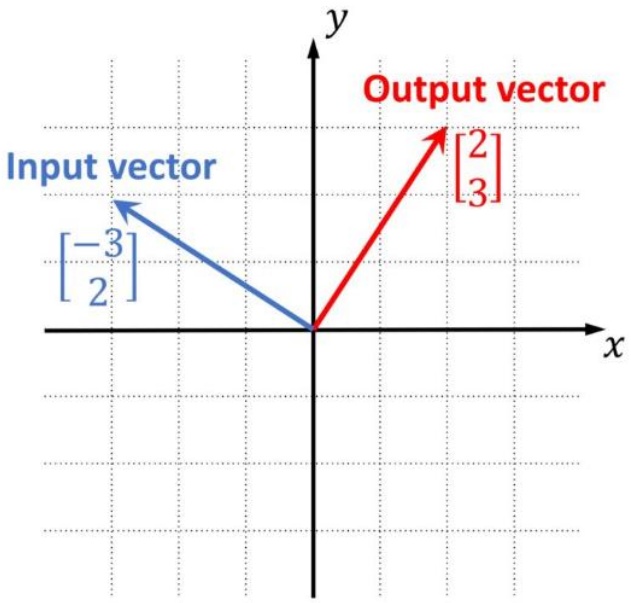
How would you describe one of these numerically?

$$\begin{bmatrix} x_{in} \\ y_{in} \end{bmatrix} \longrightarrow ???? \longrightarrow \begin{bmatrix} x_{out} \\ y_{out} \end{bmatrix}$$

We have to transform it somehow to get x_{out} and y_{out}. Very interesting thing is that we can actually observe our basis vectors, which are defined here with $\hat{i}$ and $\hat{j}$.

Linear transformation and basis vectors

- In this basic example we will have an input vector that's transformed to the output vector. To be more precise we will rotate our input vector for 90 degrees clockwise.



Input vector:

$$\vec{v} = -3\mathbf{i} + 2\mathbf{j}$$

So in coordinates:

$$\vec{v} = \begin{bmatrix} -3 \\ 2 \end{bmatrix}$$

The **standard (initial) basis** is:

$$\mathbf{e}_1 = \mathbf{i} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \mathbf{j} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

After 90 degree clockwise

Transform i

$$R_{cw}\mathbf{i} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} = -\mathbf{j}$$

Transform j

$$R_{cw}\mathbf{j} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \mathbf{i}$$

Transformed Basis

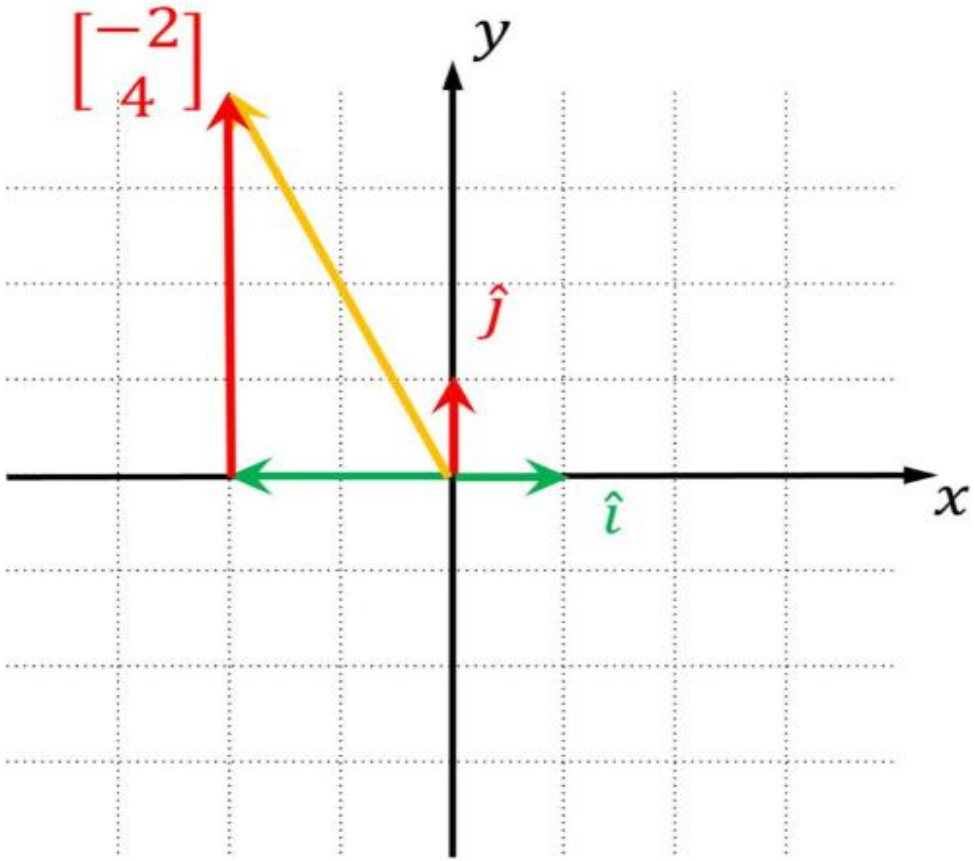
$$\boxed{\mathbf{i}' = -\mathbf{j}, \quad \mathbf{j}' = \mathbf{i}}$$

Apply the transformation:

$$\vec{v}' = R_{cw}\vec{v} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} -3 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Why is this so important? Well, let us recall that any vector is a linear combination of these two vectors. Following this concept, we can obtain coordinates of any vector. For instance,

$$\vec{v} = -2\hat{\imath} + 4\hat{\jmath}$$



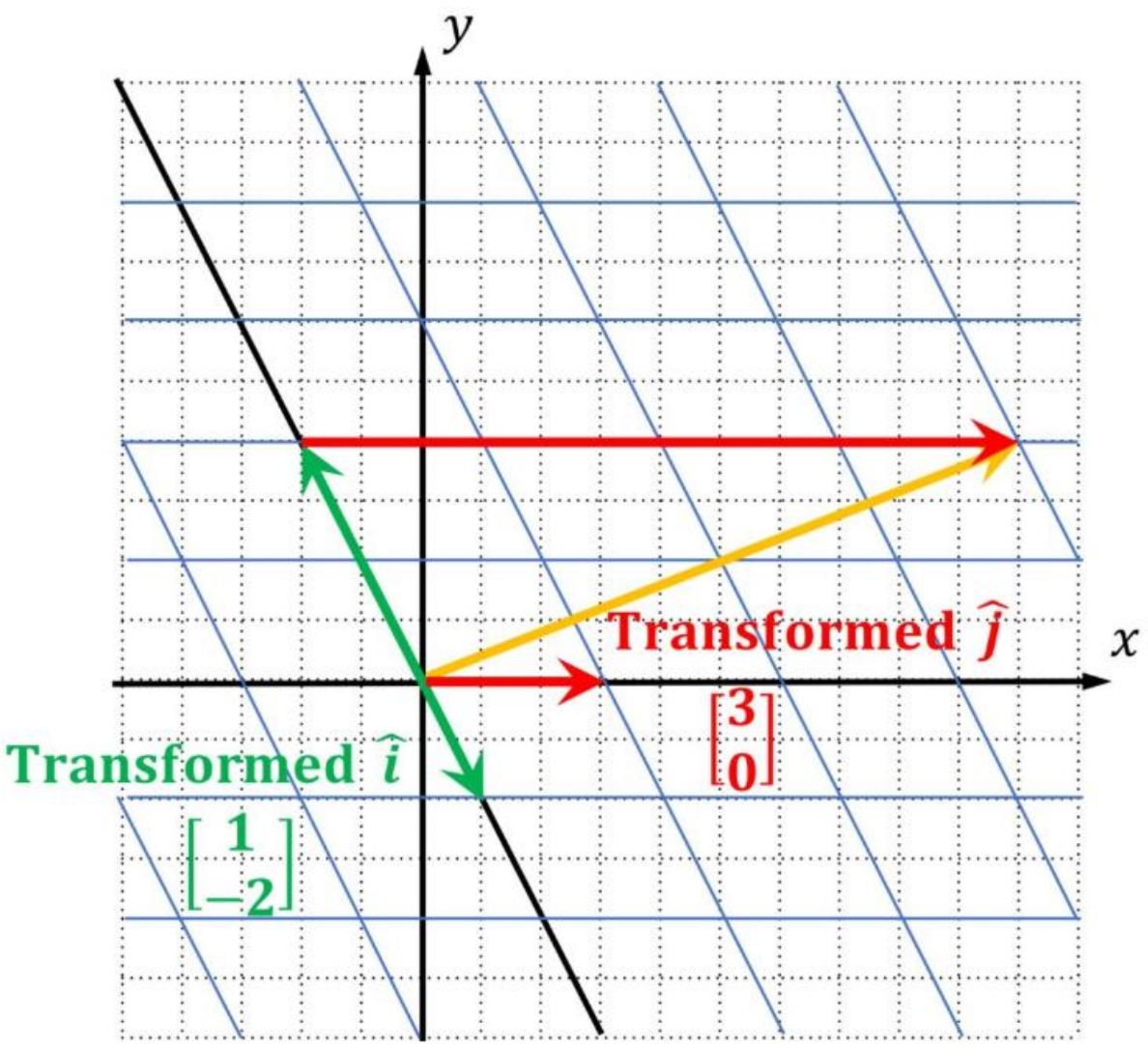
So, this is one of the examples where we have vector $\begin{bmatrix} -2 \\ 4 \end{bmatrix}$ and we have to multiply $\hat{i}$ with -2 and $\hat{j}$ with 4. Those vectors $\hat{i}$ and $\hat{j}$ are called basis vectors and they have length 1. One idea is that actually we see how each of those two vectors are transformed with our linear transformation and actually they will obtain for us transformed vector $\hat{i}$ and a transformed vector $\hat{j}$. This is illustrated in the image bellow.

So, we will observe what will happen with our original vector $(-2, 4)$ and alongside we will observe where our basis vector will be mapped for some predefined linear transformation. We will examine the transformation that maps the coordinates as illustrated in the image bellow. First of all, we can see that the transformed basis vectors are: $(1, -2)$ and $(3, 0)$. A transformed input $(-2, 4)$ vector will be at the position $(10, 4)$.

$$\vec{v} = -2\hat{\imath} + 4\hat{\jmath}$$

$$\text{Transformed } \vec{v} = -2\,(\text{Transformed } \hat{\imath}\,) + 4\,(\text{Transformed } \hat{\jmath}\,)$$

$$= -2\begin{bmatrix} 1 \\ -2 \end{bmatrix} + 4\begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix} = \begin{bmatrix} 10 \\ 4 \end{bmatrix}$$


y
x
Transformed $\hat{j}$
$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$
Transformed $\hat{i}$
$\begin{bmatrix} 1 \\ -2 \end{bmatrix}$

Matrices as the language for transformation

So basically, we will obtain the final vector as the linear combination of transformed basis vectors. We will just use the following formula to obtain the following vector $\vec{v}$ as -2 transformed vector $\hat{i}$ and transformed vector $\hat{j}$ times a scalar 4.

So, we will proceed further as follows. If we want to map our arbitrary input vector (*x*, *y*) with a linear transformation, **the output vector will preserve this pair of numbers, but they will multiply the transformed basis vectors**. To summarize we started from a vector $\begin{bmatrix} -2 \\ 4 \end{bmatrix}$ and following the simple calculation we obtained the vector $\begin{bmatrix} 10 \\ 4 \end{bmatrix}$. So, this is the vector where our input vector will be mapped.

This can also be explained with the following equations and formulas:

$$\hat{i} \rightarrow \begin{bmatrix} 1 \\ -2 \end{bmatrix} \qquad \hat{j} \rightarrow \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \end{bmatrix} \rightarrow x \begin{bmatrix} 1 \\ -2 \end{bmatrix} + y \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 1x + 3y \\ -2x + 0y \end{bmatrix}$$

A 2×2 Matrix as a linear transformation

Now, it's interesting that actually the whole transformation is defined with two transformed basis vectors, and then, we can map the whole 2-D plane if we know the transformed basis vectors. Each of these vectors is specified with just two numbers: in this case $\begin{bmatrix} 1 & 3 \\ -2 & 0 \end{bmatrix}$. Then, using these two vectors we can put them into a 2×2 matrix in a such a way that we stack these vectors along the columns and now this 2×2 matrix actually represent a useful matrix that we can use for further vector processing.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

Actually, it's just the scaling two column vectors and then summing them and this is what we get as the resulting output. This can be more intuitive way to think about the matrix-vector multiplication.

Given a transformation with the effect $\hat{\imath} \rightarrow \begin{bmatrix} -1 \\ 1 \end{bmatrix}$ and $\hat{\jmath} \rightarrow \begin{bmatrix} -2 \\ -1 \end{bmatrix}$, how will it transform the input vector $\begin{bmatrix} -3 \\ -1 \end{bmatrix}$?

- ○ $\begin{bmatrix} 5 \\ -2 \end{bmatrix}$
- ○ $\begin{bmatrix} 4 \\ -1 \end{bmatrix}$
- ○ $\begin{bmatrix} -5 \\ 2 \end{bmatrix}$
- ○ $\begin{bmatrix} 5 \\ 1 \end{bmatrix}$

Given the input $\begin{bmatrix} 3 \\ -4 \end{bmatrix}$ to the linear transformation described by the matrix $\begin{bmatrix} 1 & -6 \\ 4 & 2 \end{bmatrix}$ what vector will the transformation produce?

- $\begin{bmatrix} 28 \\ 8 \end{bmatrix}$
- $\begin{bmatrix} -22 \\ 10 \end{bmatrix}$
- $\begin{bmatrix} 27 \\ 4 \end{bmatrix}$
- $\begin{bmatrix} 22 \\ 10 \end{bmatrix}$

How matrix transformation helps ML?

$y = Wx$ y-> output vector , x-> input vector, W->Transformation matrix

This matrix:

- **rotates**
- **scales**
- **shears**
- **reflects**

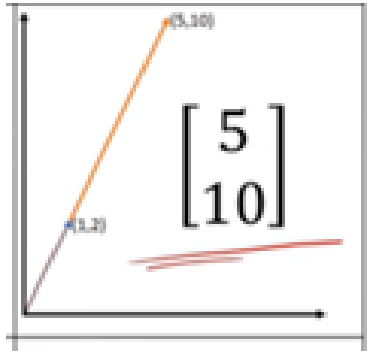
the feature space.

- **Learning = finding the right transformation that links x and y**

Eigen values and Eigen vectors

- "If a vector changes **only in length but not in direction** after transformation, it is called an **eigenvector**."
- "Think of the matrix **A** that transforms vectors."

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$



$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

$$Ax = \begin{bmatrix} 5 \\ 10 \end{bmatrix}$$

- It takes an input vector **x** and gives a new vector **Ax**.

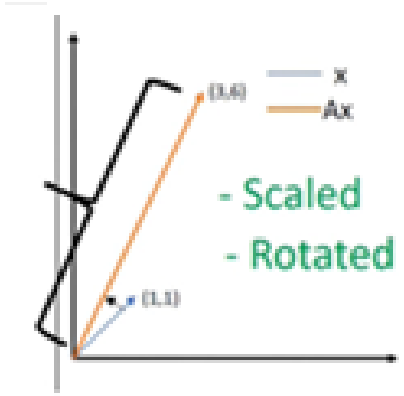
$$Ax = 5x$$

- "The number by which the eigenvector is stretched is called the **eigenvalue**."
- Eigenvector → [1,2]
- Eigenvalue → 5

- Contrast with non-eigenvector

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- After transformation:
  - Direction changes
  - Vector rotates + scales



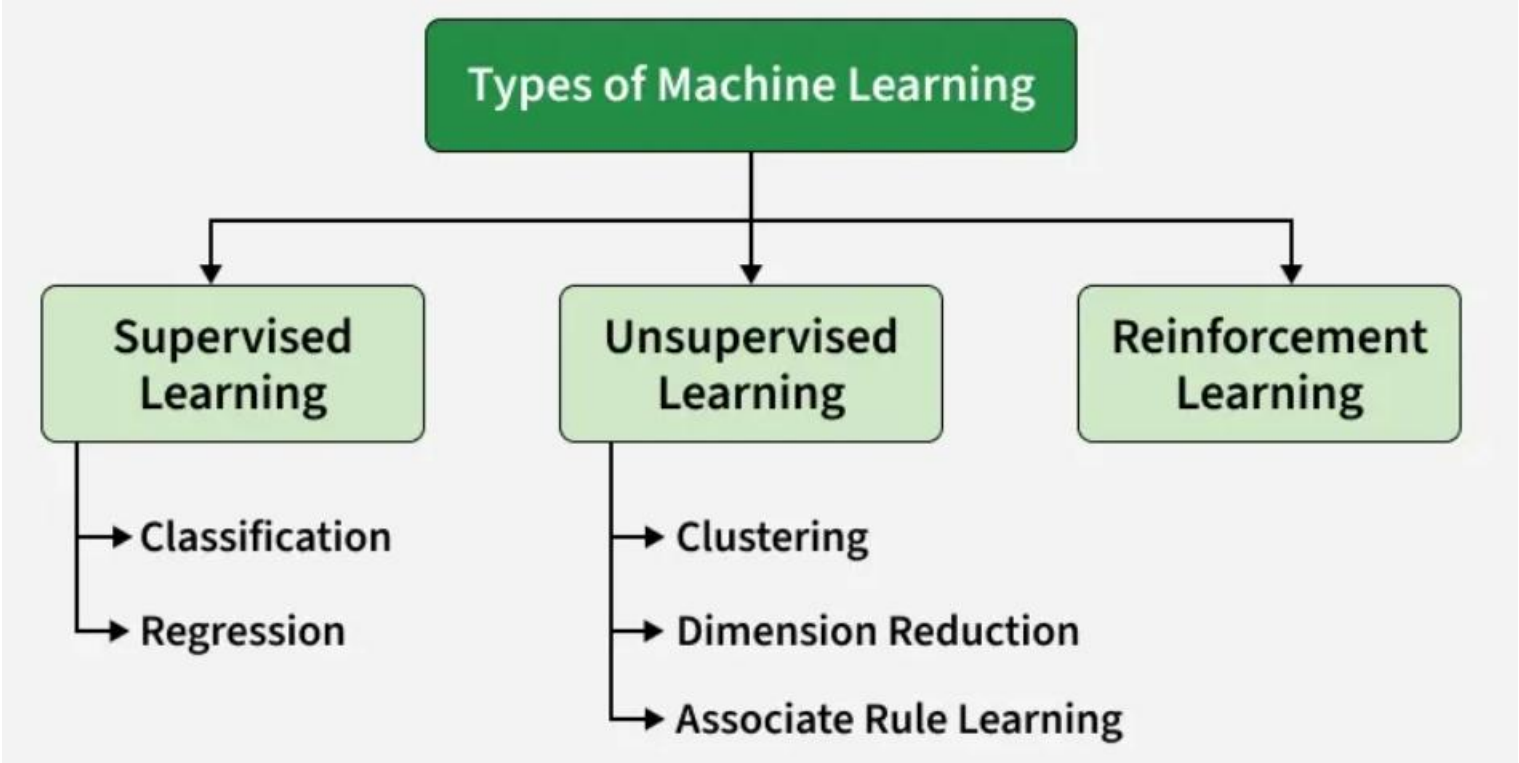
Not an eigenvector

- Linear transformations are **everywhere** in machine learning. They help:

- **Structure data** to make it usable for models.
- **Optimize models** by extracting meaningful patterns.
- **Improve efficiency** through fast matrix computations

- **Structuring Data:** Converting Words into Vectors
- **Optimizing Models:** Learning Meaningful Relationships
- **Making Computations Efficient:** Speeding Up NLP Models
  - Reduce dimensionality
  - Improve efficiency
  - Enhance learning

Paradigms of learning



Supervised

- **Supervised** machine learning models (Predictive) are trained with labeled data sets, which allow the models to learn and grow more accurate over time.
- Supervised machine learning is the most common type used today.

- **Regression algorithms**—predict output values by identifying linear relationships between real or continuous values. Regression algorithms include linear regression, random forest and gradient boosting, as well as other subtypes.

- **Classification algorithms**—predict categorical output variables (e.g., “junk” or “not junk”) by labeling pieces of input data. Classification algorithms include logistic regression, k-nearest neighbors and support vector machines (SVMs), among others.

- **Random forest algorithms**—predict a value or category by combining the results from a number of decision trees.

Unsupervised

- In **unsupervised** machine learning (Descriptive), a program looks for patterns in unlabeled data. Unsupervised machine learning can find patterns or trends that people aren’t explicitly looking for.

- **K-means clustering**—assigns data points into K groups, where the data points closest to a given centroid are clustered under the same category and K represents clusters based on their size and level of granularity. K-means clustering is commonly used for market segmentation, document clustering, image segmentation and image compression.

- **Hierarchical clustering**—describes a set of clustering techniques, including agglomerative clustering—where data points are initially isolated into groups and then merged iteratively based on similarity until one cluster remains—and divisive clustering—where a single data cluster is divided based on the differences between data points.

- **Probabilistic clustering**—helps solve density estimation or “soft” clustering problems by grouping data points based on the likelihood that they belong to a particular distribution.

Reinforcement learning

- **Reinforcement** machine learning trains machines through trial and error to take the best action by establishing a reward system.
- To deploy reinforcement learning, an agent takes actions in a specific environment to reach a predetermined goal.
- The agent is rewarded or penalized for its actions based on an established metric (typically points), encouraging the agent to continue good practices and discard bad ones. With repetition, the agent learns the best strategies.